Rendre une usine logicielle visible, vivante et pilotable

Du code à un jumeau numérique logiciel interactif

Document de cadrage — Contexte initial, concepts, schémas, technologies et proposition d'architecture

1. La demande initiale et le pourquoi de cette recherche

Le point de départ est une difficulté d'ingénierie : lorsqu'une architecture complexe est intégralement décrite en code, il devient difficile d'en saisir intuitivement les composants, les relations, le comportement pendant l'exécution et les points d'intervention possibles.

Le cas concret envisagé est une « usine » capable de produire automatiquement des applications web à partir d'une description textuelle du besoin utilisateur. Cette usine comprend de nombreux agents IA qui analysent les besoins, arbitrent et prennent des décisions, ainsi qu'une factory logicielle codée, conçue comme un mécanisme déterministe qui assemble des composants applicatifs. Dans cette vision, l'IA ne doit pas être le constructeur imprévisible de l'application finale : elle oriente et spécifie, tandis que le code réalise l'assemblage vérifiable.

La question fondamentale n'est donc pas simplement : « Comment dessiner un workflow ? » Elle est : « Comment représenter l'architecture du code, voir réellement cette architecture fonctionner et, idéalement, agir sur le système depuis cette représentation ? »

Temporal et n8n ont été cités comme repères, mais ils ne couvrent qu'une partie du besoin : l'orchestration. Réduire le problème à ces produits masquerait l'objectif plus large de visualisation, d'observabilité et de contrôle de toute l'usine.

2. Le vocabulaire technique à retenir

Concept	Fonction et portée
Software Digital Twin	Représentation dynamique d'un système logiciel réel et de son état.
Live Architecture Visualization	Visualisation de la structure et des comportements observés à l'exécution.
Visual Control Plane	Interface de commande sécurisée des composants et des opérations.
Runtime Observability	Traces, états, journaux, métriques, événements et diagnostics.
Executable Architecture	Lien formel entre modèle architectural et comportement exécutable.
Execution Replay / Time-Travel Debugging	Reconstruction d'une exécution passée, si les

données nécessaires sont conservées.

Formulation synthétique proposée : « Interactive Software Digital Twin with a Visual Control Plane », ou en français « jumeau numérique logiciel interactif doté d'un plan de contrôle visuel ». Il s'agit d'une désignation descriptive, et non d'une norme ou d'un produit unique.

3. Application à l'usine de génération automatique

La demande utilisateur traverse d'abord une couche décisionnelle IA, puis une chaîne déterministe de construction, validation et déploiement. La visualisation doit couvrir ces deux domaines sans les confondre.

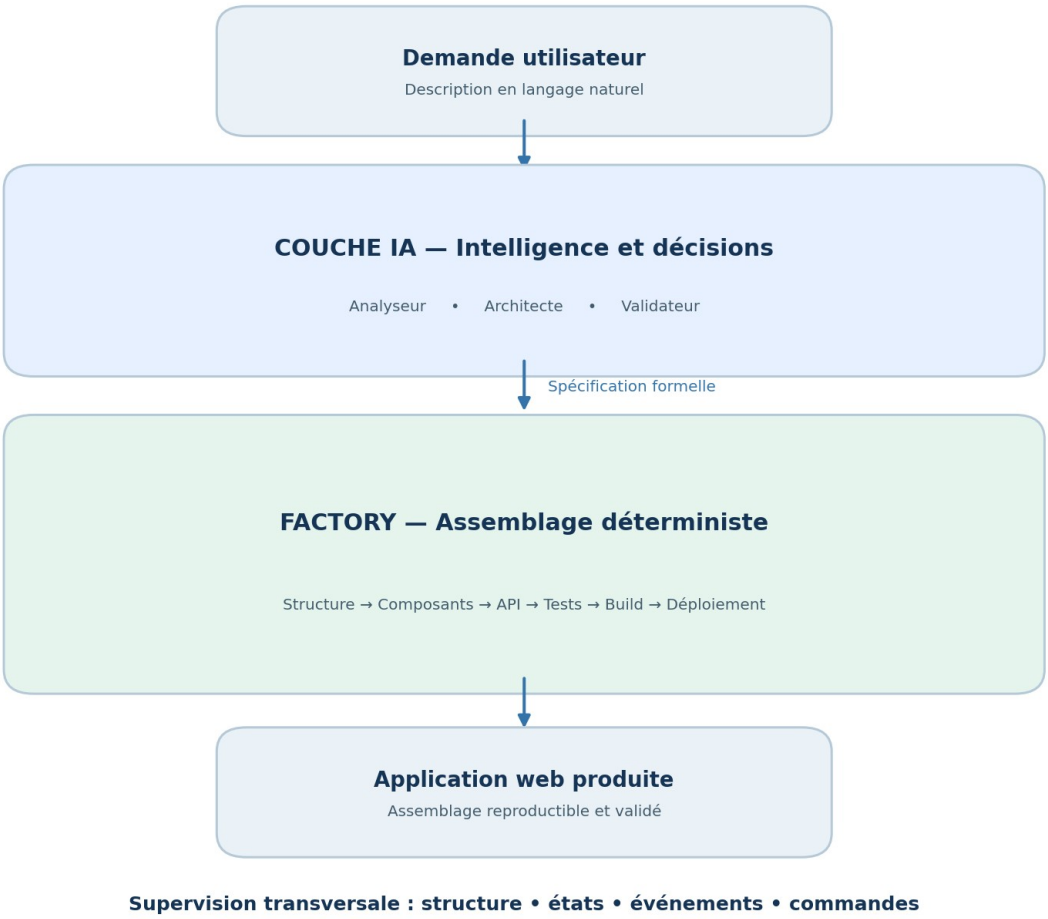


Figure 1 — Architecture fonctionnelle de l'usine et périmètre de supervision.

Exemple de navigation : sélectionner l'agent Architecte pour inspecter ses entrées, ses sorties et les justifications explicitement produites ; sélectionner ensuite le module d'assemblage pour consulter les composants retenus, les tests, l'état de l'opération et les erreurs éventuelles. Les traces rendent visibles les actions observables, pas une pensée interne cachée de l'IA.

4. Les trois niveaux complémentaires



Figure 2 — De la vue structurelle au pilotage opérationnel.

- Niveau 1 — Architecture statique : inventaire des agents, modules, API, services et dépendances. Question : de quoi l'usine est-elle constituée ?
- Niveau 2 — Architecture vivante : cheminements, opérations, décisions consignées, échanges et états actualisés. Question : que fait-elle à cet instant ?
- Niveau 3 — Architecture interactive : suspension, reprise, validation, configuration et relance par des commandes encadrées. Question : comment intervenir sans manipuler directement son code ?

Un quatrième niveau peut compléter l'ensemble : l'historique des exécutions et le replay, afin d'examiner a posteriori une fabrication et de reproduire de manière contrôlée un scénario. La reproductibilité dépend des versions, des entrées, des effets externes et du niveau d'historisation.

5. Pourquoi un orchestrateur ne suffit pas

Un orchestrateur décrit ou exécute des processus ; il ne fournit pas nécessairement un modèle complet des modules logiciels, des dépendances métier, de leurs contrats, de la structure du code ou des mécanismes de diagnostic d'un système distribué. L'usine combine au moins quatre plans distincts : structure, déroulement, observation et commande.

Une visualisation valable ne peut donc pas être un simple diagramme décoratif ni un canvas où tout le code devrait être réécrit sous forme de nœuds. Elle doit rester liée à la réalité de l'architecture et de son exécution.

6. Écosystème technologique : des briques complémentaires

Technologie	Rôle pertinent	Limite
React Flow / xyflow	Graphe interactif, navigation, nœuds et panneaux.	Ne découvre ni n'exécute seul la factory.
Backstage	Catalogue des composants, API et dépendances.	Pas un miroir complet du runtime.
Langfuse	Traces des appels IA, outils,	Couvre surtout l'observabilité

	coûts et latences.	IA.
OpenTelemetry	Instrumentation standardisée des traces, métriques et logs.	N’est pas une interface complète de commande.
Grafana Tempo	Consultation et corrélation des traces distribuées.	N’est pas un modèle métier interactif.
Temporal	Workflows durables, reprises et états d’exécution.	Ne remplace pas le catalogue architectural.
n8n	Orchestration visuelle d’automatisations.	Ne constitue pas une représentation exhaustive du code.

Ces solutions peuvent contribuer à l’ensemble, sans qu’aucune ne doive imposer la structure interne de l’usine. Le choix exact dépend du langage, de la granularité des composants, des événements disponibles et des contraintes de sûreté.

7. Architecture cible recommandée

Principe fondamental : le code demeure la source de vérité pour les comportements exécutés ; la vue est une projection instrumentée et interactive, alimentée par des états et des événements réels. Les commandes remontent par une API maîtrisée, plutôt que de modifier arbitrairement le runtime.

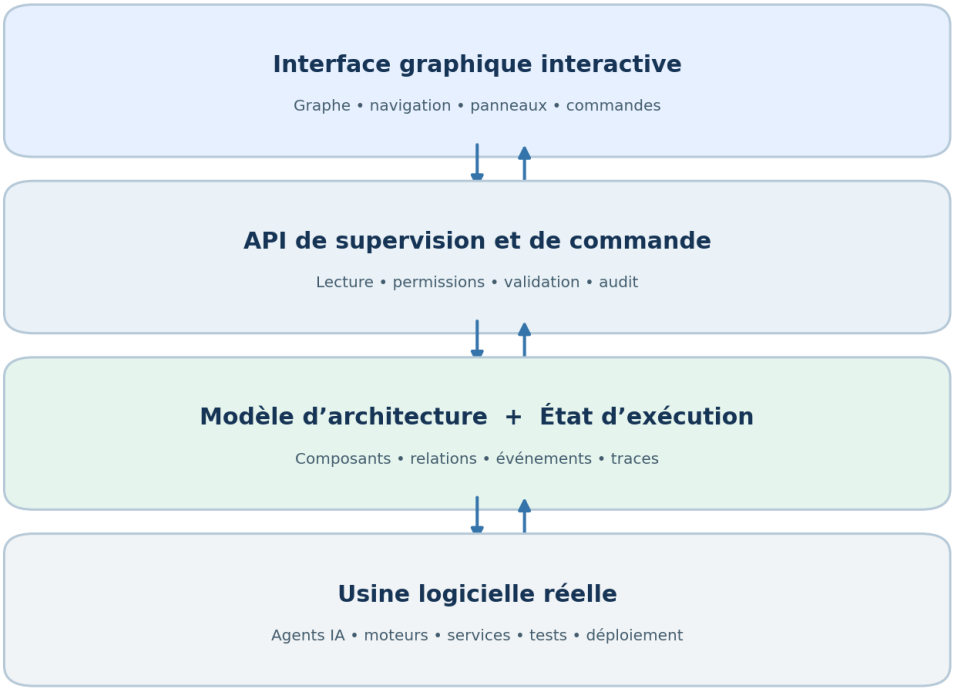


Figure 3 — Séparation interface, contrôle, modèle et exécution.

- Interface : représentation graphique, recherche, zoom, inspection, filtres, navigation et actions autorisées.
- API de supervision/commande : interrogation du système, authentification, autorisations, validation de transition, audit et accusés de réception.
- Modèle d'architecture : registre versionné des composants, identifiants stables, liens, contrats, capacités et responsabilités.
- Projection runtime : événements et traces corrélés à chaque demande, état courant, décisions consignées, erreurs et métriques.
- Factory réelle : agents IA, moteurs déterministes, assemblage, tests, stockage d'artefacts, intégrations et déploiements.

8. Navigation multiscalaire plutôt qu'un graphe géant

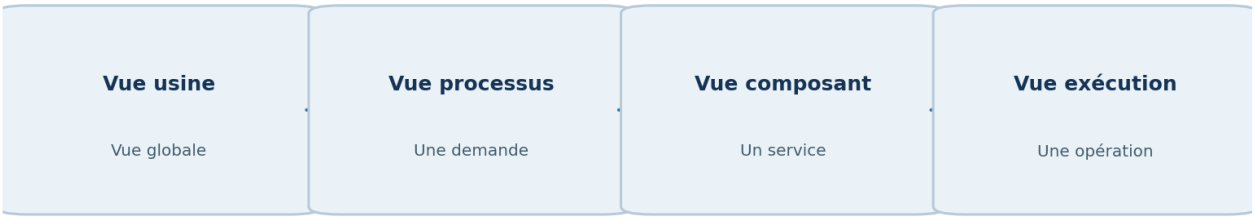


Figure 4 — Exploration progressive de l'usine vers une exécution précise.

Vue usine : état des lignes de fabrication et des grands sous-systèmes. Vue processus : progression d'une demande utilisateur. Vue composant : étapes internes d'un agent ou d'un module. Vue exécution : paramètres, événements, traces, sorties, erreurs et commandes permises. Chaque niveau doit préserver la corrélation avec les autres.

9. Les enjeux d'ingénierie à résoudre

9.1. Correspondance entre le code et le modèle visuel

La découverte automatique de fichiers, classes et appels ne suffit pas à reconstituer une architecture métier intelligible. Il faut définir une ontologie de composants, des identifiants stables et des contrats ; l'analyse statique et l'instrumentation peuvent ensuite enrichir ce modèle. Les liens doivent pouvoir être rattachés à un module, à une version et, lorsque pertinent, à une localisation du code.

9.2. Synchronisation fiable avec le runtime

L'état affiché doit être alimenté par des événements observables, horodatés et corrélés, avec règles de gestion des événements tardifs, doublons et interruptions. Une vue « temps réel » est une projection de l'état connu ; elle ne garantit pas à elle seule une vision parfaite ou instantanée du système distribué.

9.3. Commande bidirectionnelle et sécurité

Les interactions doivent être exprimées sous forme de commandes métier explicites : suspendre, reprendre, réessayer, approuver, annuler ou modifier une configuration prévue à cet effet. Elles exigent autorisations, préconditions, journal d'audit, gestion des échecs et prise en compte des opérations non réversibles. Un simple clic graphique ne doit jamais produire une mutation arbitraire.

9.4. Déterminisme de la factory

L'objectif est de limiter la variabilité là où la reproductibilité est essentielle : composants versionnés, schémas formels, plans d'assemblage validés, règles de compatibilité et tests automatiques. Les agents peuvent recommander ou sélectionner, mais la chaîne de construction applique des règles explicites et vérifiables.

10. Stratégie pragmatique de mise en œuvre

- Étape 1 — Définir les entités du modèle : agents, services, modules, étapes, artefacts, ressources, relations et états.
- Étape 2 — Donner à chaque composant un identifiant stable et exposer un manifeste de sa structure et de ses capacités.
- Étape 3 — Instrumenter les exécutions avec événements corrélés et traces de bout en bout ; lier chaque fabrication à un identifiant de demande.
- Étape 4 — Construire une vue statique navigable, puis y projeter les états et événements live.
- Étape 5 — Ajouter progressivement des commandes sûres, gouvernées par une API et des règles de transition.
- Étape 6 — Introduire l'historique, les comparaisons de versions et le replay contrôlé lorsque nécessaire.

11. Conclusion et décision structurante

L'ambition n'est pas de « faire du n8n en plus grand », mais de bâtir un jumeau numérique interactif de l'usine logicielle : un espace unique reliant architecture, événements, opérations, décisions observables et commandes sûres. L'approche privilégiée est code-first, instrumentée, pilotée par événements et structurée autour d'un modèle explicite des composants.

Décision clé : faut-il rétroconcevoir la visualisation à partir d'une factory existante, ou concevoir dès l'origine une factory qui expose nativement ses composants et ses événements ? La seconde voie est plus structurante et généralement plus fiable ; la première exige davantage d'analyse et d'instrumentation rétroactive.

Cette note constitue un cadrage de conception, non la spécification exhaustive d'un produit ni une preuve de faisabilité détaillée.